

COLEXIO VIVAS S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Ejercicios

Ejercicio 1

a) Realiza un servidor multihilo de fecha y hora. Aceptará los siguientes comandos desde cualquier cliente (Protocolo):

- **time:** Envía hora, minutos y segundos,
- **date:** Envía día, mes y año.
- **all:** Envía tanto la fecha como la hora.
- **close password:** Junto con el comando close se debe verificar que viene una contraseña. Si esta es correcta el servidor ha de finalizar y se lo indica al cliente. Si no devuelve un mensaje de error al cliente (Debe diferenciarse el error de contraseña no válida o que no se haya enviado la contraseña).

La contraseña estará guardada en un archivo de texto en %PROGRAMDATA% denominado password.txt con una sola línea conteniendo la contraseña en texto plano.

En cuanto se envíe el dato pedido, se cerrará la conexión (No hay bucle de atención al cliente en el hilo del cliente).

b) Haz un cliente simple en modo gráfico con cuatro botones (uno por comando) y un TextBox para meter una contraseña y una Label para mostrar el resultado y probarlo.

El cliente no se mantiene conectado, es decir, cada vez que se pulsa un botón, se conecta, se envían y reciben datos y se desconecta. Fíjate que todos los botones van a hacer casi lo mismo por lo que debes minimizar el código repetido.

También tendrá la posibilidad de indicarle la IP y puerto de conexión (aunque venga con unos predefinidos) en un formulario de diálogo.

El cliente no se bloqueará en ningún momento.

COLEXIO VIVAS S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Ejercicio 2

Haz un chatroom. El servidor permite conectarse a cuantos clientes deseen y lo que recibe de un cliente lo reenvía al resto de los clientes indicando la IP.

Para realizarlo debes crear una colección de clientes conectados que sea común a todos los hilos. La colección se recomienda hacerla de objetos de una clase definida por ti donde se incluya, por lo menos, nombre de usuario, IP y StreamWriter.

Cuando un cliente se conecte, el servidor pide nombre de usuario al nuevo cliente.

Además el servidor informará al resto de clientes de quién se conecta y quién se desconecta cuando esto suceda.

Una vez conectado, cualquier cosa que escriba un cliente se envía al resto de los usuarios.

Cuando un usuario recibe un mensaje, este viene precedido de **usuario@IP** de quién lo envía.

Existirán dos comandos que no se envían al resto de usuarios cuando un cliente los escriba:

#exit: Permite que un usuario se desconecte.

#list: El cliente recibe la lista de usuarios.

Por supuesto debe ser robusto ante salidas abruptas de un cliente.

Como cliente usa directamente telnet.

(Opcional)

Realiza un cliente gráfico. Ten en cuenta que necesitas trabajar con tareas distintas para poder enviar y recibir en cualquier momento desde el cliente.

También de forma opcional puedes ampliar el servidor de forma que admita varias salas de chat, usuarios registrados, etc...

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Ejercicio 3 (Ejemplo examen)

Se desea realizar un servidor de turnos. Crea en un nuevo proyecto de consola una clase denominada **ShiftServer** en la cual estarán definidos los siguientes miembros:

- Dos listas de usuarios: por un lado un vector de usuarios posibles denominado **users** (vector de strings) y **waitQueue** donde se añaden los usuarios que están en lista de espera (colección de strings).
- Función **ReadNames** la cual lee de un archivo de texto, cuya ruta está indicada en el parámetro, nombres de usuarios separados por punto y coma (;). Puede tener varias líneas. Rellena el vector *users* con dicha lista de nombres.
- Función **ReadPin** la cual lee de un archivo de texto, cuya ruta está indicada en el parámetro, los 4 primeros caracteres y los devuelve como int. Si hay un problema con el archivo o el pin devuelve -1.
- Habrá una función Init la cual inicia el servidor multihilo con las especificaciones siguientes:
 - El puerto de espera será el 31416. Si este puerto estuviera ocupado busca el primero libre a partir del 1024. Si llegara al último válido la aplicación finaliza. Muestra en pantalla el puerto de conexión.
 - Luego cargará la lista de usuarios válidos mediante la función *ReadNames* del archivo *usuarios.txt* que se encuentra en el directorio indicado por *%userprofile%*.
 - Una vez que el cliente se conecta se lanza el hilo que controla a dicho cliente. Lo primero es que este recibe un mensaje de bienvenida y se le pide nombre de usuario. Comprueba si dicho nombre está en la lista o si es la palabra admin. Si no cumple lo anterior simplemente se desconecta informando de "Usuario desconocido".
 - Si es admin se le pide un pin. El servidor obtiene el pin mediante la función *ReadPin* pasándole un archivo denominado *pin.txt* que se encuentra también en *%userprofile%*. Si hay algún problema al leer el archivo la contraseña será 1234. Si la contraseña no es correcta, lo

COLEXIO VIVAS S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

desconecta directamente.

- Una vez conectado, para un usuario normal los comandos posibles son:
 - **list**: Envía al cliente el listado de alumnos en espera (waitQueue).
 - **add**: Añade el usuario actual al final de la lista. Debes concatenarle al nombre la fecha y hora de conexión. Envía al cliente un mensaje de OK. No puede haber usuarios repetidos en esta lista. Puedes usar un separador (dos puntos, guion,...) entre nombre e información temporal para aplicar un split.

En cuanto el usuario mete uno de estos dos comandos, ya se finaliza la conexión (no se queda en bucle a la espera de otro comando).

- En el caso de ser el usuario admin, además de las anteriores dispone de los siguiente comandos:
 - **del pos**: borra el usuario indicado por la posición pos. Si hay alguna incorrección envía el mensaje delete error al cliente.
 - **chpin pin**: guarda pin en el archivo pin.txt en caso de que sea un pin válido (integer de 4 cifras). Indica al cliente si ha sido guardado o si ha habido un error.
 - **exit**: Sale del servidor. Al revés que un cliente "normal", el administrador no es desconectado tras meter un comando si no que solo finaliza la conexión mediante este comando (o shutdown).
 - **shutdown**: Finaliza el servidor de forma natural. Al terminar el servidor, waitQueue debe ser guardada en algún archivo (el tipo que quieras: text, binario o incluso en algún formato estándar como json o xml). En la siguiente ejecución, se cargarán estos datos.

El Main estará en otra clase y simplemente creará un objeto de la clase ShiftServer y ejecutará su función principal.

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Ejercicio 4 (Se validan por separado cliente y servidor)

Ahorcado en red. Se trata de realizar un servidor de palabras y récords y un cliente de juego.

a) Servidor

Dispone de acceso a un archivo de palabras (archivo de texto con palabras separadas por comas). Cuando lea el archivo pasa las palabras a mayúsculas.

También mantiene un archivo de récords, que será un archivo binario donde se guarda una lista con 3 objetos tipo Record (clase sencilla que almacena nombre de 3 caracteres y cantidad de segundos)

Se dispone del siguiente protocolo de comandos que puede recibir el servidor:

gw: GetWord. El servidor envía una palabra aleatoria al cliente de la lista que tiene.

sw palabra: SendWord. El cliente le envía una palabra nueva al servidor. Este la guarda en el archivo y la añade a la colección de palabras actual. El servidor envía un OK o ERROR si se ha hecho todo bien o hubo un problema respectivamente.

gr: GetRecords. El servidor le envía la lista de récords del archivo al cliente.

sr record: SendRecord. El cliente le envía un nuevo récord al servidor. Si el tiempo es menor que alguno que los que tiene almacenado lo guarda y, si fuera necesario porque ya hay tres, deshecha el peor. Si no fuera menor que ninguno lo deshecha. El servidor responde con un ACCEPT o REJECT dependiendo si ha guardado o no el récord.

close clave: cierra el servidor si se dispone de la clave adecuada.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

b) Cliente

El cliente al ejecutarse pedirá una palabra al servidor. El cliente cuando acaba la partida envía el tiempo en segundos que le ha llevado adivinar la palabra y un nombre de 3 caracteres.

El resto del juego será como de costumbre: en el cliente, tras comenzar un juego nuevo pedirá una palabra al servidor y usará varias labels para colocar los subrayados (o letras cuando la acierte). No se puede hacer en una única label. Aparecerá un fragmento de dibujo del ahorcado usando el componente a medida del tema de DI.

Acuérdate de medir el tiempo (hazlo mediante un Timer) y pedir el nombre para el récord solo si va a entrar en la tabla de récords. También debe permitir ver la tabla de récords que se le pedirá al servidor.

Habrà también un formulario específico de envío de palabras. Se permitirán dos posibilidades, o enviar una única palabra o enviar un archivo de palabras. El archivo serán palabras separadas por comas.

Crea un interfaz adecuado con todas las opciones en un menú.

Si ya lo habías hecho en el tema de componentes, puedes ampliar dicho programa y hacerlo online o comenzar uno nuevo.

(Opcional)

Busca la manera de disponer en el servidor de un archivo con una lista de contraseñas válidas. Estas contraseñas estarán encriptadas (de estilo de archivo passwd de linux). También el envío de la contraseña desde el cliente se encriptará. Busca alguna librería o método para realizar esta parte.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM
	MÓDULO	Programación de servicios y procesos		
	PROTOCOLO:	Apuntes clases	AVAL:	DATA:
	UNIDAD COMPETENCIA			
			CURSO:	2º

Ejercicio 5 (Ejemplo examen)

A principios del S.XXI la corporación FlyRell llevó la evolución de las moscas a la fase Flyxus. Un ser casi idéntico al insecto original conocido como un replicantfly. Los replicantfly 6 eran superiores en fuerza y agilidad a sus congéneres biológicos y encima con una picadura de mil demonios. Fueron declaradas ilegales en la tierra salvo en Guantánamo y en las nuevas ediciones de Gran Hermano (vaya usted a saber con qué objetivo). Las fugas existían y surgió un escuadrón especial de la policía denominado Fly Runners. Pero era un trabajo duro y cuando les soltaron por ahí a matar moscas estos dijeron a sus superiores: "Mira bro, sin entrenar yo paso de perseguir bichos".

Las instancias superiores de fuerzas y seguridad del estado encargaron un sistema completo de entrenamiento en red para los Fly Runners a los fabulosos alumnos del Colegio Vivas, bien sabida su legendaria capacidad de codificación que dejarían a Neo de noob total. Así que, bro, esto es lo que tienes que hacer:

Se desea realizar un servidor de moscas, para ello se creará un nuevo proyecto de consola en Visual Studio con las siguientes características:

Habrá una clase denominada **FlyRunner** la cual tendrá las siguientes características:

- Propiedad **Sw** de tipo StreamWriter.
- Propiedad **KilledFlies** de tipo int. Representa el número de moscas matadas por un FlyRunner.
- Propiedad **Bites** de tipo int. Representa el número de picaduras recibidas por un FlyRunner.
- Constructor con un único parámetro tipo StreamWriter que inicialice sw. Los int los inicializa a 0.

Otra clase denominada **FlyServer** con los siguientes miembros:

- Colección de objetos tipo FlyRunner que representará a los clientes conectados. Se denomina **clients**.
- Método **GetPort()**: función que trata de leer un archivo binario (mira el

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

apéndice en el tema de archivos) con números enteros denominado *port.bin* situado en el directorio indicado por la variable de entorno *userprofile*. Debe leer dos números enteros seguidos que corresponden a un puerto principal y uno secundario.

Si el archivo no existe, hubiera algún problema o los números son valores no válidos para puertos, creará un archivo binario con los valores 135 y 31416 como enteros en las dos primeras posiciones y luego escribe 8 números enteros más aleatorios entre 1024 y el valor máximo posible de puerto (escribe todo como int). En este caso la función devuelve -1.

Si ha cargado bien los dos puertos, devolverá el primero de ellos que esté libre (haz comprobación de puerto ocupado aquí). Si ninguno lo estuviera devuelve -1.

- Método **InitServer()**: Este método tiene una estructura clásica de servidor multihilo. Obtiene un puerto mediante GetPort, luego se queda a la escucha en cualquier interfaz de red en el puerto indicado por la función. Si esta función devuelve -1 el servidor finaliza.

Si entra un cliente, lo lanza en un hilo aparte llamando a la función FlyRunnerThread definida más abajo. Tras esto se queda a la escucha de un nuevo cliente.

- Método **FlyRunnerThread**. Representa al hilo de atención al cliente y recibe como parámetro un **object** que será de tipo Socket. Dentro del mismo se realizan las tareas típicas de inicialización de los distintos Streams y añadir un objeto tipo FlyRunner con el StreamWriter actual a la colección *clients*. Tras esto entra en el bucle de comandos que sigue el siguiente protocolo:
 - El servidor le envía una mosca, es decir, el mensaje **fly**, y se queda a la espera de que el cliente responda. Realiza un switch con la recepción de mensajes cuyos casos pueden ser:
 - Si el cliente manda el mensaje **fsw** (abreviatura de flyswatter: pala matamoscas) puede suceder una de tres cosas de forma aleatoria (todo con la misma probabilidad):
 - Posibilidad 1: Mata a la mosca, es decir, incrementa el contador de moscas (KilledFlies) de su objeto FlyRunner (fr) e informa con el

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

mensaje: `$"Killed {fr.KilledFlies} fly/flies!!"`

- Posibilidad 2: La mosca le da un mordisco con lo cual incrementa el contador de picaduras de su objeto FlyRunner (fr) e informa con el mensaje: `$"You have been bitten. Number of bites: {fr.Bites}."`
- Posibilidad 3: La mosca escapa y pica a otro flyrunner de forma aleatoria (es decir, selecciona uno aleatorio de la colección distinto al actual para incrementar su nº de picaduras). Además se le envía el mensaje: `"Other fly bites you!!"`
- Si el cliente manda un mensaje **quit** (Terminar) entonces se le informa de los mordiscos recibidos y de las moscas matadas. Además se le indican al resto de los clientes que ha salido alguien mediante el siguiente mensaje (fr es el objeto actual, el que ejecutó el quit) : `$"Someone leaves with {fr.Bites} bites and {fr.KilledFlies} flies killed."`

Tras este mensaje el hilo terminará de forma natural.

- Si el cliente sale abruptamente (pulsando el icono X en la consola, por ejemplo) el hilo terminará también de forma natural y con el mensaje al resto de los clientes tal y como se explica en "quit". Evita repetir código mediante goto case, por ejemplo. Tanto en este caso como en el de "quit" el cliente será eliminado de la colección clients.
- Finalmente si hay un error en el comando (es decir, no se escribe nada de lo indicado anteriormente), la mosca le pica dos veces (incrementa en 2 el contador de picaduras) y el mensaje sera: `$"Big mistake, you were bitten twice. Number of bites: {fr.Bites}."`. Y además le quitará una picadura a otro FlyRunner aleatorio (puedes estar incluido) y se le manda el mensaje `"I get you one bite!!"`.

Revisa bien las variables comunes entre varios hilos para un correcto uso de los lock.

En la clase **Program** estará el Main que simplemente llama a InitServer de un objeto tipo FlyServer.

	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Ejercicios Opcionales

Op 1

Realiza un servidor Web simple. Lo único que hará es servir páginas html/htm estáticas si las hay y dar el mensaje de error si no existen. Observa en: http://es.wikipedia.org/wiki/Hypertext_Transfer_Protocol el Ejemplo de diálogo. Como mínimo tiene que responder a peticiones GET. Si quieres amplía el protocolo.

Op 3

Realiza un programa que realice un ping por programación a una IP que se le indique. Puedes hacerlo en modo consola para que funcione como comando. Debes informarte sobre el protocolo ICMP y los sockets en modo RAW para poder hacerlo.